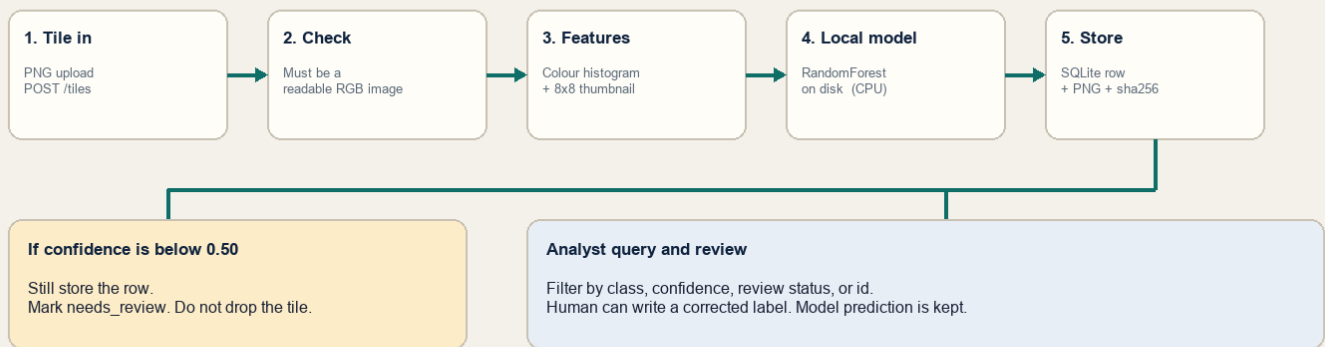


GalaxEye | Offline tile classification | Design

Part 1 · How a tile moves through the system

Core path the brief asked for



What I chose, and why

- Model: local RandomForest on their candidate_tiles. No OpenAI / cloud vision. Interface can swap to ONNX later.
- Store: SQLite + files. Offline, one file to copy. Full scores + bytes, not only the top class.
- Unsure predictions: persist + flag. Dropping them is silent data loss. Pretending we are sure is worse.
- Query: structured filters, not a map product. The brief asked for a thin slice.
- Eval CSV is a scorecard only. Training labels are folder names. Predict uses pixels, never the filename.

Assumptions and questions: see page 2 of DESIGN.pdf · Scenario answers: PART3.md

Design note — offline tile classification

GalaxEye take-home, Part 1. Informal, 1–2 pages. The required slice is ingest → classify → store. Query, review, and canary are the smallest extensions that make the design honest.

What I am building

An air-gapped service that takes a satellite tile, runs a local land-use classifier, writes a structured result, and lets an analyst ask simple questions of those results. No hosted model APIs. If the box has no internet, inference still works.

Tile flow:

1. Analyst or upstream ingest POSTs a PNG to `/tiles`.
2. The API validates it is a readable RGB image.
3. A frozen local model returns a class, a confidence, and the full score vector.
4. If confidence is below a threshold, the row is stored as `needs\_review` rather than silently treated as truth.
5. The original bytes are kept on disk; metadata goes to SQLite.
6. An analyst lists by label, review status, or confidence, or opens one id.

Still not in scope (and not asked): login, Postgres, GPU/CNN, map search, multi-process workers, cloud. Batch ingest, analyst table, human review write-back, and a local canary are implemented.

Components

Piece Choice Why

API FastAPI One obvious endpoint, typed, easy to change live in an interview

Model sklearn RandomForest on colour + coarse spatial features CPU, seconds to train, no GPU, no download at runtime

Store SQLite + tile files on disk Offline, one file to copy, SQL is enough for “show me Forest tiles”

Runtime One Python process Matches an isolated box; no Redis/Postgres/S3 to stand up

I treated the classifier as a swap point, not the product. Tomorrow it can be a fine-tuned ResNet. Today it is a small model that actually runs.

Decisions I weighed

1. What “querying the results” means

Options I considered:

- **A.** Dump a CSV after each run. Fine for a lab, useless once two people ask different questions.
- **B.** Full search (text, map, embeddings). Too much product for a slice.
- **C.** Structured filters on stored predictions: label, confidence, review status, time, id.

I chose C. The questions an analyst actually asks first are “how many industrial tiles this week?”, “what was this id labelled?”, and “what is sitting in the review pile?”. If GalaxEye later needs “tiles like this one” or lat/lon, that is a new index on the same records, not a rewrite.

2. What to store

I store more than the top class:

- predicted label and confidence
- full class scores (so we can change the threshold later without rerunning)
- model version
- review status
- original filename and a copy of the bytes

I do not store only the argmax. If results look wrong a month later, I want to reopen the exact image and the exact score vector the model produced, not re-guess from a label.

I do not store raw feature vectors in v1. They help debugging, but they couple the database to one featurizer. Easy to add.

3. Predictions the model is not sure about

Options:

- Always write the top class and pretend we are sure.
- Drop low-confidence tiles. Silent data loss. Bad.
- Separate “prediction” from “decision”: persist everything, mark `needs\_review` below a threshold (0.50 here).

I chose the third. The threshold is a product decision, not a model decision. A 0.51 Forest vs 0.49 River should not ship as confident land-use. The API still returns a label so the pipeline never blocks; the review flag is what an analyst filters on.

I would rather tune that threshold on a labelled eval set (they gave us one) than invent a fancy abstain model on day one.

4. Model: train vs pretrained, sklearn vs torch

The zip is a EuroSAT subset with 1,050 labelled candidate tiles and a 210-tile eval set. ImageNet weights do not know `AnnualCrop` vs `SeaLake`. A hosted vision API is disallowed.

So: train locally on `candidate\_tiles`, measure on `eval\_set`, ship the artifact. Accuracy is not the grade, but a model that cannot tell water from forest would make the rest of the design hard to talk about.

Why not PyTorch here: heavier install, slower CPU train, harder to modify live, and the brief said a wrong model is fine. A RandomForest on colour histograms + an 8×8 thumbnail is honest about the constraint (offline, CPU) and landed at ~77% on their eval set. Highway is the weak class (confused with Residential and River), which is exactly the kind of error a review threshold is for. If this were a real GalaxEye box I would drop a fine-tuned MobileNet/ONNX behind the same `LandUseClassifier` interface.

5. SQLite vs a “real” database

SQLite wins for an isolated appliance: no daemon, one file, trivial backup. It loses if we have many concurrent writers or multi-GB imagery catalogues. The first thing that would force a move is ingest volume, not query complexity. Postgres later; same schema.

Assumptions

- Tiles are single 64×64-class RGB PNGs, one land-use label each, EuroSAT-like.
- “Offline” means inference and storage. Installing Python deps once on a connected laptop is allowed; the running service does not call the network.
- One tile per request is the core path. Batch can be a loop over that path.
- We are not given geolocation, sensor, or capture time, so I do not invent them.
- Analysts can live with label / confidence / review filters at first.
- A model that is often wrong is acceptable if the system is honest about it.

Questions I would ask

1. What is the downstream decision? Map product, change detection, or a human report? That sets how costly a Forest↔River swap is vs a Residential↔Industrial swap.
2. Do tiles arrive one-by-one or as a strip/scene we must cut ourselves?
3. Is there a human review step in ops today, and who owns the labels?

4. Will production imagery stay Sentinel-2 / EuroSAT-like, or will sensor and GSD change? That decides whether this featurizer survives.
5. What is the latency budget, and is CPU-only a hard constraint on the target box?
6. Do we need to keep the raw tile indefinitely (forensics / retraining) or only the prediction?
7. Who is allowed to query, and is this box shared or single-operator?

Built after the thin slice

- Daily-style canary: `POST /canary` rewrites `var/canary.json`.
- Review write-back: `PATCH /tiles/{id}/review` keeps the model label and stores the human one.
- Batch ingest: multi-file upload and `POST /ingest/eval-set`.
- Analyst table + review screen in the local UI.
- Append-only log at `var/app.log`.

Still later, if this were a real box: per-class thresholds, two model artifacts side by side, geo index.